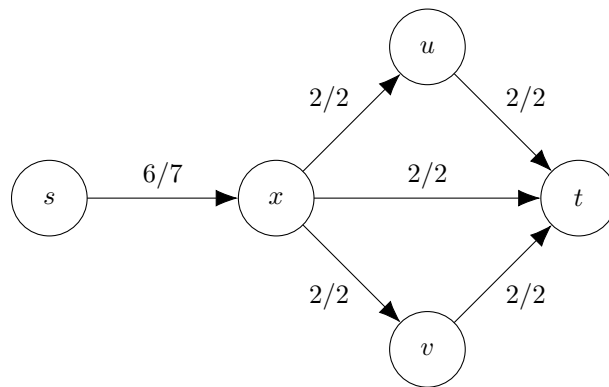


DAA Assignment 3

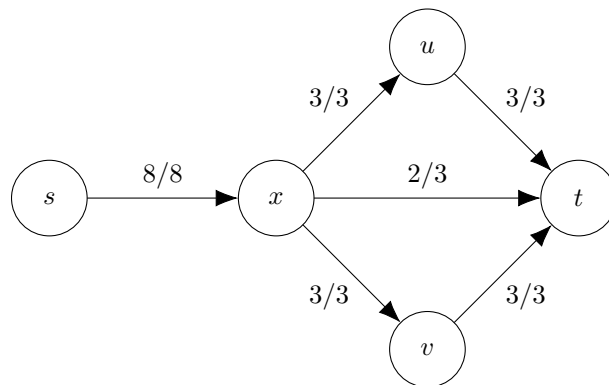
Abhiram Aravind [Sr No 25238]

1 KT Chapter 7 Question 5

The statement is false. Here is a counter example:



Here, by the max-flow min-cut theorem, the minimum cuts are $\{xu, xv, xt\}$ and $\{ut, vt, xt\}$. However, if you increase all capacities by one, the minimum cut changes.



Here, the minimum cut is $\{sx\}$, which is different.

2 KT Chapter 7 Question 12

Input: Directed graph G , k .

Output: Set of edges of size k , whose removal minimises the maximum flow of the resulting graph.

Since the capacities of each edge is 1, then removing k edges from the minimum cut should suffice.

1. Find the maximum flow f of the graph G using the Ford-Fulkerson method.
2. Calculate the residual graph G' for G and flow f .
3. Perform a DFS on G' from the vertex s and mark all vertices reachable from s .
4. Find the min-cut: the set of edges adjacent to exactly one marked vertex.
5. Take any subset of the min-cut with k edges. Removal of these edges reduces the flow by k .

Each edge has a capacity of 1. Therefore, removal of any single edge reduces the flow by at most 1, and by induction, removal of k edges can reduce the flow by at most k . By the max-flow min-cut theorem, all the edges as found in step 4 constitute a min-cut. By removing edges from the min-cut, we guarantee that the flow would reduce by 1 for each edge. If it doesn't, that would mean that there is another path from s to t through which we could direct a flow of 1, which contradicts the fact that we have a min-cut. If the size of the min-cut is less than k , we can completely cut off all flow from s to t .

Runtime Complexity

- Ford-Fulkerson method with unit capacities on edges: $O(|E||V|)$
- Calculating residual graph: $O(|E|)$
- DFS to mark vertices reachable from s : $O(|E|)$
- Finding min-cut: $O(|E|)$

Total: $O(|E||V|)$

3 JE Chapter 10 Question 13

(a) Directed Graph

Input: Directed graph G , vertices s and t .

Output: Bottleneck edge from s to t in G .

Constraint: Time complexity $O(|E| \log |V|)$

We perform a modified Dijkstra's algorithm but extracting the vertex with the largest weight of the incoming edge instead of the shortest path to it. What it does is expand a directed tree beginning at s , taking the maximum capacity edge at each moment. When reaching t , we stop. The optimality of Dijkstra's algorithm guarantees that all other paths from s to t have edges with capacities no larger than that of the bottleneck of the path found. If there was a path with a larger bottleneck, then all of the other edges would have larger capacities than the bottleneck and Dijkstra's algorithm would have used that path to reach t instead.

1. Set $S = \emptyset$. This the set of already visited vertices.
2. Create empty priority queue Q .
3. $Q.\text{Add}(\text{item}:s, \text{priority}: 0)$;
4. Set $d(s) = 0$. $d(v)$ is the largest weight of the incoming edge to v .
5. Set $p(s) = \text{NULL}$. $p(v)$ refers to the parent vertex of v in the path from s to v .
6. While $Q \neq \emptyset$
 - (a) $u = Q.\text{ExtractMax}()$;
 - (b) If $u = t$, terminate loop.
 - (c) Set $S = S \cup \{u\}$
 - (d) For all edges $u \rightarrow v$:
 - If $u \notin Q \wedge u \notin S$:
 - i. $Q.\text{Add}(\text{item}: v, \text{priority}: c(u, v))$;
 - ii. Set $d(v) = c(u, v)$;
 - iii. Set $p(v) = u$;
 - Else if $u \in Q \wedge d(u) < c(u, v)$
 - i. $Q.\text{Update}(\text{item}: v, \text{priority}: c(u, v))$;
 - ii. Set $d(v) = c(u, v)$;
 - iii. Set $p(v) = u$;
7. Begin at t and follow the parents $p(t)$ to construct the path from s to t . Return the minimum capacity edge along this path.

Since this is essentially a modified Dijkstra's algorithm, with a fibonacci heap, it has a time complexity of $O(|E| + |V| \log |V|)$.

(b) Undirected Graph

Input: Undirected graph G , vertices s and t .

Output: Bottleneck edge from s to t in G .

Constraint: Time complexity $O(|E| + |V|)$

We observe that if $e = \text{bottleneck}_G(s, t)$, then all edges in the path from s to t along e has capacities no smaller than $c(e)$. Furthermore, for other paths with smaller bottlenecks, by definition they have edges with capacities smaller than $c(e)$. Thus, we can say that $e = \text{bottleneck}_G(s, t)$ is the edge with the largest capacity such that s and t remain connected when all edges of capacities less than $c(e)$ is removed. We can easily find e using **QuickSelect** with Median of Medians approach in $O(|E|)$.

Define $\text{BK}(G', s, t)$ as the recursive algorithm that runs on a graph G' as follows:

1. Let E be the edges in G' . Pick a pivot e in E with median capacity $c(e)$.
2. Partition E around pivot e . Let R be the right half (set of edges with capacities greater than or equal to $c(e)$).
3. Perform a depth first search from s using only edges in R .
4. Perform a depth first search again from s using only edges in $R \setminus \{e\}$.
5.
 - If t is reachable from s in the first search but not the second, return e as the bottleneck.
 - If t is reachable from s in both searches, delete all edges of G' not in R and return $\text{BK}(G', s, t)$.
 - If t is not reachable from s in either search, contract all edges in R of G' and return $\text{BK}(G', s, t)$.

$\text{BK}(G, s, t)$ returns the maximum bottleneck as required.

The contractions and edge removals are necessary to keep the algorithm linear. One can safely remove edges less than the maximum bottleneck as it does not affect the path with the maximum bottleneck. Similarly, one can safely contract edges greater than the maximum bottleneck as it preserves the nature of paths. Since the edges are undirected, any path entering a component can leave via any other vertex in it. We first note that the algorithm terminates: s is connected to t when no edges are removed, and s is disconnected from t when all edges are removed. Thus, there exists an edge e where all edges with capacities greater than or equal to $c(e)$ is insufficient to connect s to t , but along with e is enough to do so. And the correctness of **QuickSelect** ensures that this edge is found.

Runtime complexity:

- Finding median capacity: $O(|E|)$
- Partition: $O(|E|)$
- Depth first search without e : $O(|E|)$
- Depth first search with e : $O(|E|)$
- Removing edges in not in R : $O(|E|)$
- Contracting edges in R : $O(|E|)$

$$T(|E|) = T\left(\frac{|E|}{2}\right) + O(|E|)$$

$$T(|E|) \in O(|E|)$$

This does not work for directed graphs as one cannot contract edges while preserving paths. Doing the algorithm without contracting edges leads to worst case $O(|E| \log |E|)$.

4 JE Chapter 10 Question 14

Increment(e)

Input: Flow network G from source s to sink t , max-flow f , edge e .

Output: Max-flow after e 's capacity is increased by 1.

All one has to do after incrementing the capacity is run a single iteration of the Ford-Fulkerson algorithm.

1. If $c(e) > f(e)$, increment $c(e)$ and return f as-is.
2. Else increment $c(e)$ by 1 and perform a DFS starting at s and find a path P from s to t where for each edge w , $c(w) > f(w)$. If no such path exists, return f .
3. Set $f'(w) = f(w) + 1$ for all $w \in P$ and $f'(w) = f(w)$ for all $w \notin P$. Return f' .

Since the algorithm uses a single instance of DFS, followed by recomputing f' once for each edge, the whole algorithm runs in $O(|V| + |E|)$. We observe that f' thus computed is indeed correct. When one increases the capacity of an edge by 1, the max flow cannot increase by more than 1. If it could, then that would mean the original max-flow could be increased as well.

Decrement(e)

Input: Flow network G from source s to sink t , max-flow f , edge e .

Output: Max-flow after e 's capacity is decreased by 1.

If the capacity of e is greater than the flow through e , then $c(e)$ can be reduced without affecting the max-flow f . Otherwise, we reduce the flow along one path that goes through e and run one iteration of the Ford-Fulkerson algorithm.

1. If $c(e) > f(e)$, decrement $c(e)$ by 1 and return f as-is.
2. Else decrement $c(e)$ by 1. Suppose $e = uv$. Perform DFS to find a path P_u from s to u whose edges have non-zero flow.
3. Now perform DFS again to find a path P_v from v to t whose edges have non-zero flow. Let $P = P_u \cdot uv \cdot P_v$.
4. Set $f'(w) = f(w) - 1$ for all $w \in P$ and $f'(w) = f(w)$ for all $w \notin P$.
5. Run one iteration of Ford-Fulkerson: using DFS, find a path from s to t in the residual graph. If one exists, increase the flow f' through the edges in the path by 1.
6. Return f' .

Since the algorithm uses two instances of DFS, followed by a single iteration of Ford-Fulkerson, the whole algorithm runs in $O(|V| + |E|)$. We observe that f' thus computed is indeed correct. When one decreases the capacity of an edge by 1, the capacity constraint for that edge could be violated, hence we find a path from s to t and reduce its flow. When we reduce its flow, it could be the case that another path not through e could now accommodate 1 more unit of flow, so one more iteration of Ford-Fulkerson might be necessary to compute the actual maximal flow f' .